DispatchGuard — Hints

> Read the story and try to build it first. These hints point you at the pieces and the easy-to-miss parts, without giving the whole answer. The full code is in the complete solution.

Hint 1 — the pieces, and keeping each reviewer narrow

The same shape as CodeGuard: a routing Supervisor, three narrow reviewers, a compile step. Keep each job in its own file:

- `src/message.py` — `parse_message(text) -> {type, body}`. The type is a leading `[internal]` / `[transactional]` / `[marketing]` tag (default `marketing` if none).
- `src/schemas.py` — `Finding(severity, category, message)`.
- `src/agents/safety_reviewer.py` — `run_safety(message)`: regex over the body for leaked PII, internal markers, risky claims. **Deterministic** — that's what makes it testable.
- `src/agents/brand_reviewer.py` — `run_brand(model, collection, message)`: retrieve the relevant voice-guide rules (RAG), then ask the model which lines break which **retrieved** rule.
- `src/agents/completeness_reviewer.py` — `run_completeness(message)`: heuristics (promo → needs end date; every blast → needs an unsubscribe line).
- `src/agents/supervisor.py` — `choose_reviewers`, `decide_next`, `compile_review`, and the `supervisor` node.
- `src/rag/` (store/ingest/retriever), `src/state.py`, `src/graph.py`.

Build order: Safety on a dirty *and* a clean message → Brand with RAG → Completeness → Supervisor + graph.

Hint 2 — the routing table, and where the value lives

The whole point is routing by message type and compiling. Settle this table first:

```
choose_reviewers(message):
    reviewers = ["safety"]              # safety always runs
    if type == "internal":      return reviewers          # safety only
    if type == "transactional": reviewers += ["completeness"]
    if type == "marketing":     reviewers += ["brand", "completeness"]
    return reviewers
```

```
compile_review(findings):
    dedup by (category, message)
    sort by {high:0, medium:1, low:2}
    empty -> "DispatchGuard: cleared to send."
```

The graph is `START -> supervisor`; each reviewer node runs its reviewer and returns `Command(goto="supervisor", update={findings+=, done+=})`; the supervisor sends control to the next pending reviewer, or to `compile -> END`.

Two things make this real, not a formality: (1) the Safety scanner is tested on a **clean** message too (a scanner only tested on bad input flags good copy), and (2) the Brand reviewer's prompt says **only use the rules provided** and the rules came from retrieval — so a finding can only cite a rule that's actually in your voice guide. Put one **specific, unusual** rule in that guide (e.g. "never open with 'Hey'") so retrieval has something to prove. The whole file set is in the complete solution.